

Arquitetura e Organização de Computadores

Aula-03: Função e Interconexão

Eliseu César Miguel

Livro: Arquitetura e Organização de Computadores
William Stallings 8ª Edição
Universidade Federal de Alfenas

December 17, 2021

Organização da Aula

- 1 Componentes do Computador
- 2 Função do Computador
- 3 Estrutura de Interconexão e Barramentos

Organização da Aula

- 1 Componentes do Computador
- 2 Função do Computador
- 3 Estrutura de Interconexão e Barramentos

Organização da Aula

- 1 Componentes do Computador
- 2 Função do Computador
- 3 Estrutura de Interconexão e Barramentos

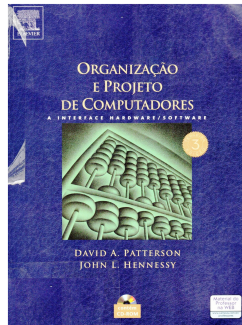
Bibliografia básica



Livro Texto



Complementar



Complementar

Além da bibliografia básica, visite o programa de ensino para ver outras bibliografias. Também, durante o curso, várias bibliografias em sítios da Internet serão apresentadas.

Introdução: Material Adicional



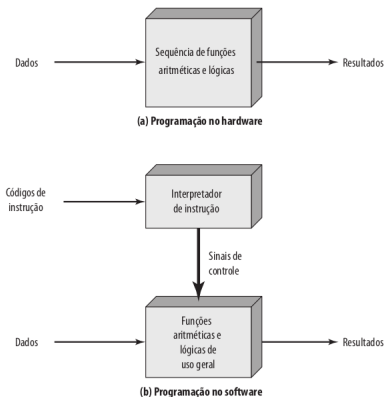
Material de uso exclusivo do professor

Componentes do computador

Hardwired & Software: As duas abordagens podem ser combinadas!

Hardware e Software são logicamente equivalentes

Figura 3.1 Abordagens de hardware e software



Hardwired: mais seguro e inflexível

Software: mais vulnerável e flexível

Componentes do computador

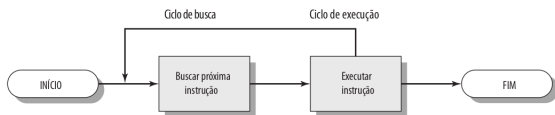
Conceito de Programa

Um programa de computador é um conjunto de instruções que descrevem uma tarefa a ser realizada por um computador. O termo pode ser uma referência ao código fonte, escrito em alguma linguagem de programação, ou ao arquivo na forma executável deste código fonte¹.

Esse conceito pode ser visto por outros pontos de vista!

- conjunto de instruções a serem realizadas
 - ▶ a semântica define a programação desejada
 - ▶ hardware deve oferecer unidades básicas à execução das instruções
 - ▶ a execução do programa basea-se na repetição do ciclo de instrução
- a unidade de controle pode ser *hardwired* ou microprogramada
- um conjunto de sinais lógicos devem ser gerados para executar as instruções

Figura 3.3 Ciclo de instrução básico



¹ https://pt.wikipedia.org/wiki/Programa_de_computador

Função do computador: Exemplo

Consideremos uma máquina hipotética com formato de instruções e dados:

Figura 3.4 Características de uma máquina hipotética



(a) Formato de instrução



(b) Formato de inteiro

Contador de programa (PC) = Endereço da instrução
Registrador de instrução (IR) = Instrução sendo executada
Acumulador (AC) = Armazenamento temporário

(c) Registradores internos da CPU

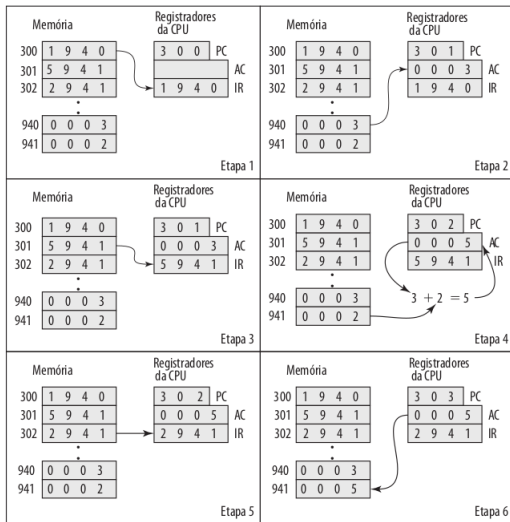
0001 = Carrega AC da memória
0010 = Armazena AC na memória
0101 = Adiciona da memória ao AC

(d) Lista parcial de opcodes

- Instruções nas classes:
 - ▶ carga e armazenamento
 - ▶ lógica e aritmética
 - ▶ controle
- Dados inteiros em sinal de magnitude (isso é bom?)
- Endereçamento *direto* à memória

Função do computador: Exemplo

Figura 3.5 Exemplo de execução de programa (conteúdo da memória e dos registradores em hexadecimal)



Classes de Paralelismo e Arquiteturas Paralelas

Taxionomia de Flynn (Década de 1960)

- *Single Instruction Single Data: (SISD);*
[categoria de uniprocessador]
- *Single Instruction Multiple Data: (SIMD);*
[multiprocessados como GPU]
- *Multiple Instruction Single Data: (MISD);*
[modelo teórico]
- *Multiple Instruction Multiple Data: (MIMD);*
[cada processador busca suas instruções e seus dados]
[clusters e multithreads de diferentes programas]



Arquitetura de Computadores: Uma abordagem Quantitativa
5ª Edição. Capítulo 01, página 09



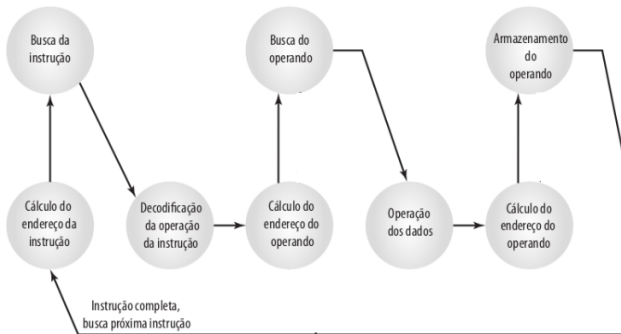
Arquitetura e Organização de Computadores
8ª Edição. Capítulo 17, página 515

Função do computador

Ciclo de instrução considerando a Taxionomia de Flynn

Figura 3.6 Diagrama de estado do ciclo de instrução

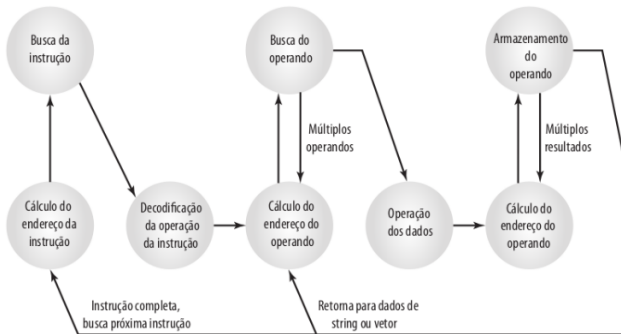
SISD



Função do computador

Ciclo de instrução considerando a Taxionomia de Flynn

Figura 3.6 Diagrama de estado do ciclo de instrução



Função do computador: Interrupções

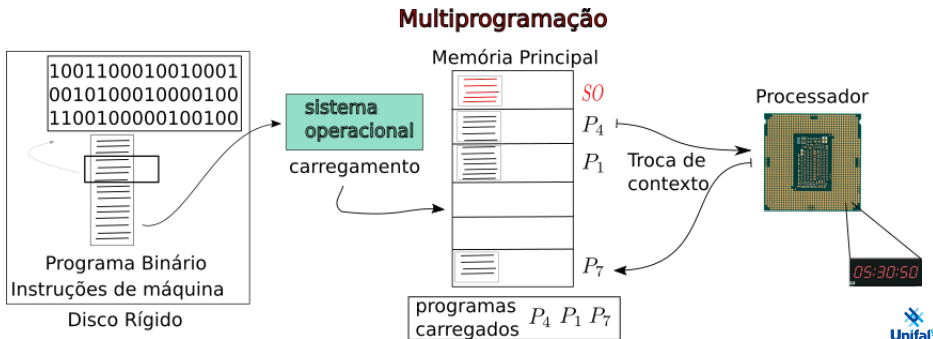
Tabela 3.1 Classes de interrupções

Programa	Gerada por alguma condição que ocorre como resultado da execução de uma instrução, como o <i>overflow</i> aritmético, divisão por zero, tentativa de executar uma instrução de máquina ilegal ou referência fora do espaço de memória permitido para o usuário.
Timer	Gerada por um timer dentro do processo. Isso permite que o sistema operacional realize certas funções regularmente.
E/S	Gerada por um controlador de E/S para sinalizar o término normal de uma operação ou para sinalizar uma série de condições de erro.
Falha de hardware	Gerada por uma falha como falta de energia ou erro de paridade de memória.

Função do computador: Interrupções

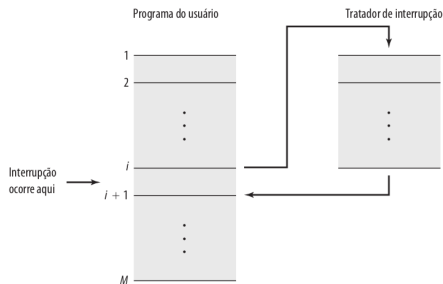
Tabela 3.1 Classes de interrupções

Programa	Gerada por alguma condição que ocorre como resultado da execução de uma instrução, como o <i>overflow</i> aritmético, divisão por zero, tentativa de executar uma instrução de máquina ilegal ou referência fora do espaço de memória permitido para o usuário.
Timer	Gerada por um timer dentro do processo. Isso permite que o sistema operacional realize certas funções regularmente.
E/S	Gerada por um controlador de E/S para sinalizar o término normal de uma operação ou para sinalizar uma série de condições de erro.
Falha de hardware	Gerada por uma falha como falta de energia ou erro de paridade de memória.



Função do computador: Interrupções

Figura 3.8 Transferência de controle via interrupções



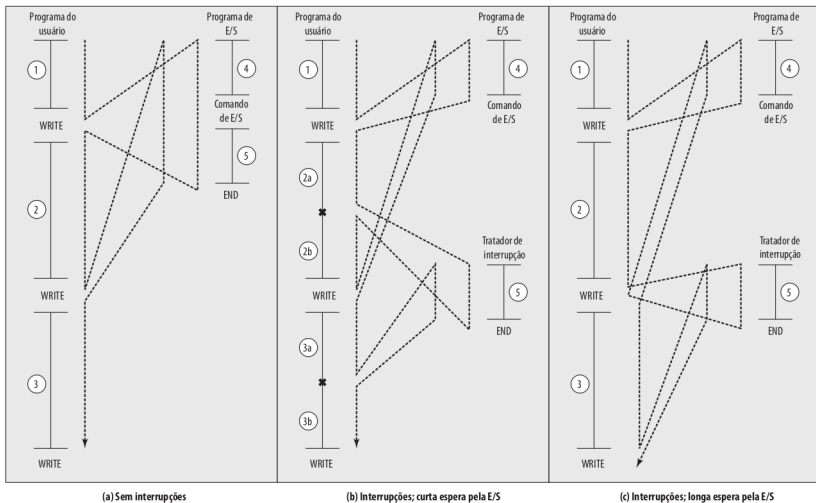
Os tratadores de interrupções podem ser implementados:

- em seu programa
- em hardware
- no sistema operacional

Função do computador: Interrupções

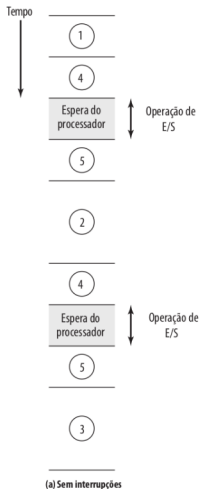
Figura 3.7

Fluxo de controle do programa sem e com interrupções

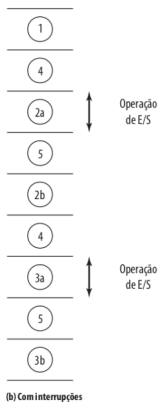


Função do computador: Interrupções

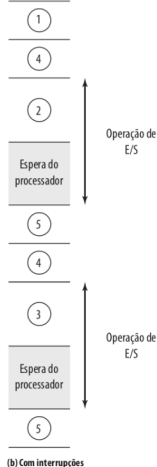
Sincronização do programa: espera curta pela E/S



Sincronização do programa: espera longa pela E/S



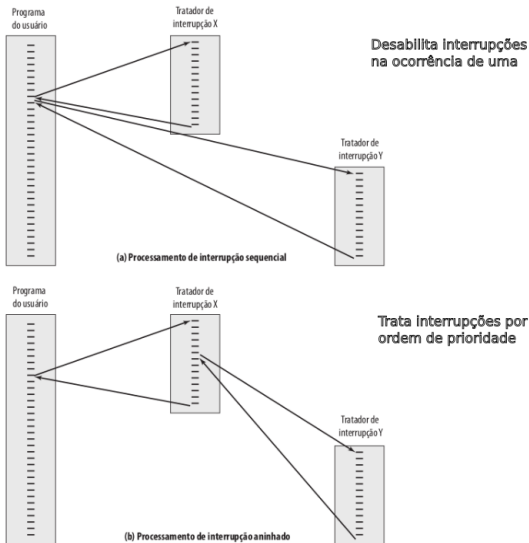
(b) Com interrupções



(b) Com interrupções

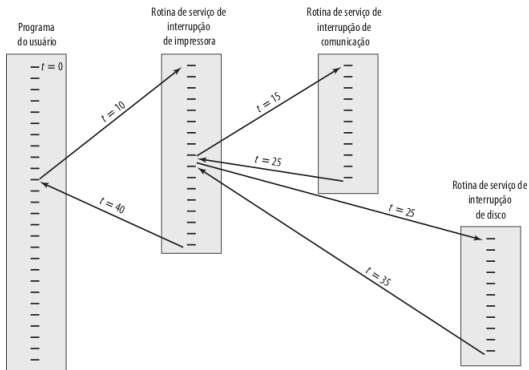
Função do computador: Interrupções

Figura 3.13 Transferência de controle com múltiplas interrupções



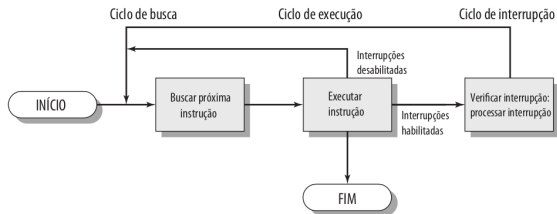
Função do computador: Interrupções

Figura 3.14 Exemplo de sequência de tempo de múltiplas interrupções



Função do computador: Interrupções

Figura 3.9 Ciclo de instrução com interrupções



Função do computador: Interrupções

Figura 3.9 Ciclo de instrução com interrupções

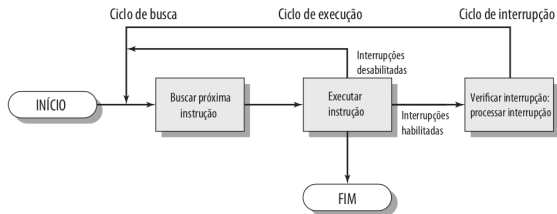
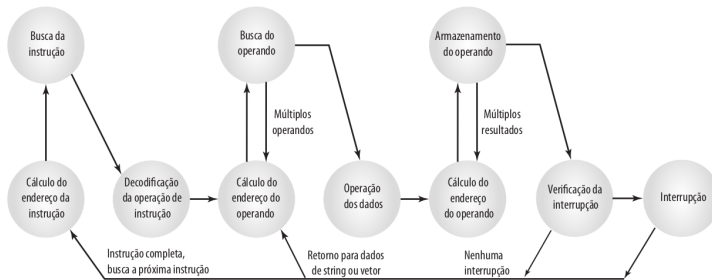


Figura 3.12 Diagrama de estado do ciclo de instruções, com interrupções



Função do computador: Entrada e Saída (E/S)

- Estudado em maiores detalhes no Capítulo 07
- Oferecem velocidades de escrita e leitura mais baixas que a Memória Principal e CPU
- Devemos distinguir entre *módulo* e *dispositivos*
- Problema: Conectar dispositivos sem perder o desempenho
- Soluções em Entrada/Saída:
 - ▶ Programada
 - ▶ Interrupção
 - ▶ Direct Memory Access (DMA)



Módulos

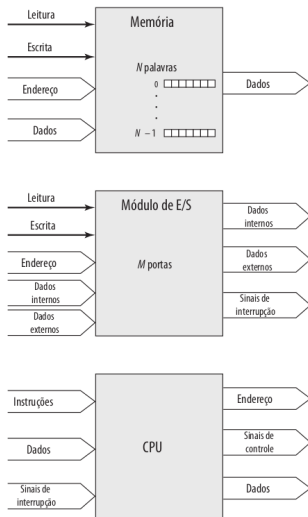


Dispositivos

Função do computador: Entrada e Saída (E/S)

Principais módulos do computador de propósito geral (não exclusivo)

Figura 3.15 Módulos do computador



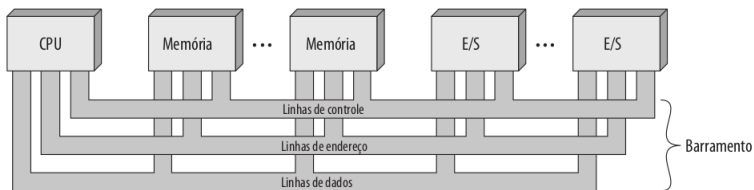
Estrutura de Interconexão e Barramentos

Um barramento é um caminho de comunicação entre dois ou mais dispositivos.

Características

- é um meio de transmissão compartilhado
- o sinal de um dispositivo é visível por todos do barramento
- somente um dispositivo pode enviar sinais em um determinado momento
- colisão: acontece quando mais de um dispositivo ocupa o barramento
- *barramento do sistema*: que liga a CPU à memória

Figura 3.16 Esquema de interconexão de barramento



Estrutura de Barramentos

Linhas de dados

- 32, 64, 128 ou mais linhas paralelas
- cada linha transporta 1 *bit*
- conhecidas como *largura do barramento*
- influenciam diretamente no desempenho

Linhas de endereços

- determinam origem ou destino dos dados
- a largura do barramento de endereços determina o limite de memória endereçável
- podem ser usadas para endereçar (dados e) portas de E/S

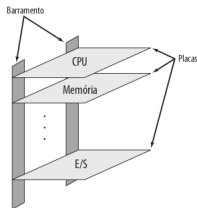
Linhas de controle

- usadas para controlar o uso das linhas de dados e endereço
- transmitem sincronização e controle entre os dispositivos
- *sinais de sincronização*: indicam a validade dos dados e endereços
- *sinais de comando*: indicam operações a serem realizadas
- *sinais de estado*: servem para o dispositivo informar seu estado

Estrutura de Interconexão e Barramentos

Quais são os desafios?

Figura 3.17 Realização física típica de uma arquitetura de barramento



Para um módulo enviar dados a outro:

- 1 obter o uso do barramento
- 2 transferir dados por meio do barramento

Para um módulo receber dados de outro:

- 1 obter o uso do barramento
- 2 transferir requisição de dados ao outro
- 3 aguardar a transmissão

- Esses passos foram simplificados. Incluímos, ainda, os envios de endereços e confirmações de recebimento com sucesso ou falha.
- Como coordenar as permissões de uso do barramento e a temporização dos sinais?
- Estudaremos várias opções para a implementação dos barramentos!

Hierarquia de Barramentos

Apenas um barramento pode não ser uma boa estratégia para conectar muitos componentes:

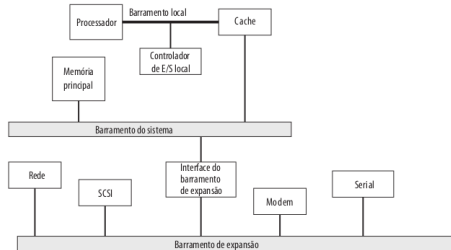
- **quanto mais dispositivos, maior o tamanho do barramento!** Isso causa grandes atrasos na propagação das informações. A grande troca de posse do barramento entre os dispositivos piora o desempenho nestes casos
- **o barramento torna-se um gargalo para a demanda de transferência!** Isso é consequência das diferentes quantidades de *bits* transmitidos/recebidos por cada tipo de dispositivo. Aumentar a largura do barramento não é totalmente viável. Outro problema é a grande variação nos tempos de acesso de cada dispositivo

Como solução, vários sistemas usam a *hierarquia de barramentos*, que permite:

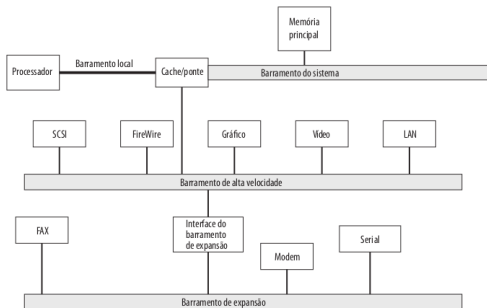
- Agrupar dispositivos com acesso mais frequente e agrupar os menos frequentes
- Criar estratégias entre os barramentos para minimizar as diferenças entre os tempos de acesso dos dispositivos
- Permitir que várias trocas de dados sejam executadas de forma intercalada

Hierarquia de Barramentos

Figura 3.18 Exemplo de configurações de barramento



(a) Arquitetura de barramento tradicional



(b) Arquitetura de alto desempenho

Implementação de barramentos: Elementos

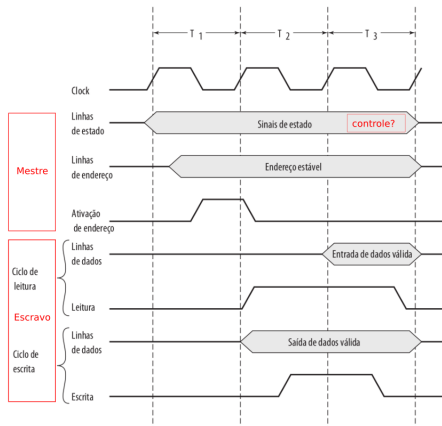
Tabela 3.2 Elementos do projeto de barramento

Tipo	Largura do barramento
Dedicado	Endereço
Multiplexado	Dados
Método de arbitração	Tipo de transferência de dados
Centralizado	Leitura
Distribuído	Escrita
Sincronização	Ler-modificar-escrever
Síncrona	Leitura-após-escrita
Assíncrona	Bloco

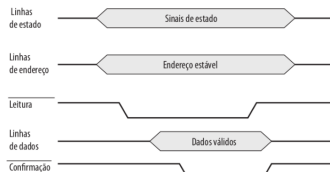
- **ler-modificar-escrever** (operação atômica de leitura seguida de uma escrita no mesmo endereço). O endereço é transmitido apenas uma vez. Serve para evitar que outro mestre escreva na posição de memória em questão. Assim, pode-se proteger os recursos em uma memória compartilhada em sistemas multiprogramados
- **leitura-após-escrita** (operação atômica de escrita seguida por leitura no mesmo endereço). Serve para verificar se a escrita foi realizada com sucesso

Temporização: Síncrona e Assíncrona

Temporização de operações de barramento síncronas



Temporização de operações de barramento assíncronas

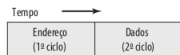


(a) Ciclo de leitura do barramento do sistema



(b) Ciclo de escrita do barramento do sistema

Figura 3.21 Tipos de transferência de dados do barramento



Operação de escrita
(multiplexada)



Operação de leitura
(multiplexada)



Operação leitura-modificação-escrita



Operação leitura-após-escrita

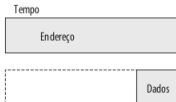


Transferência de dados em bloco



Dados e endereço enviados pelo mestre no mesmo ciclo por linhas de barramento separadas.

Operação de escrita (não multiplexada)



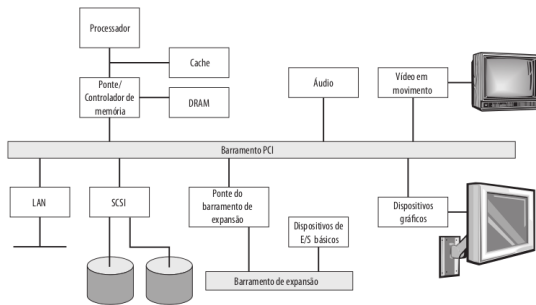
Operação de leitura (não multiplexada)

Barramento PCI (*Peripheral component interconnect*)

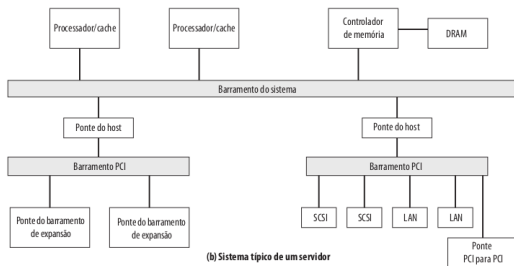
A Intel começou a trabalhar n PCI em 1990 para seus sistemas baseados n Pentium.

- Intel abriu ao domínio público. Surge a *PCI Special Interest Group*
- Funciona como *mezanino* ou *periférico*
- Bom desempenho para E/S de alta velocidade
- Grande largura de banda independente do processador
- Admite outros barramentos ligados a ele
- Temporização síncrona (com altas velocidades) e arbitração centralizada
- Permite mais de um processador (sistema multiprocessado)

Figura 3.22 Exemplo de configurações PCI



(a) Sistema típico de um computador de mesa (desktop)



(b) Sistema típico de um servidor

Exercícios

Lista de exercícios a ser divulgada no Moodle

Agradecimentos

Agradecimentos Especiais:

Agradeço a toda a comunidade $\text{\LaTeX}$.
Em especial a *Till Tantau* pelo *Beamer*.

<https://www.tcs.uni-luebeck.de/mitarbeiter/tantau/>

Desta forma, tornou-se possível a escrita deste material didático.